Лабораторная работа №5

Эмуляция и измерение потерь пакетов в глобальных сетях

Авдадаев Джамал Геланиевич

2026-11-07

Содержание (i)

1. Цели и задачи работы

2. Процесс выполнения лабораторной работы

3. Интерактивные эксперименты NETEM

4. Автоматизация сетевых экспериментов

5. Исследование коррелированных потерь

6. Исследование переупорядочивания пакетов

7. Исследование повреждения пакетов

8. Выводы по проделанной работе

1. 1. Цели и задачи работы

1.1 Цель лабораторной работы

Изучение возможностей инструмента **NETEM** для эмуляции потерь, повреждения, дублирования и переупорядочивания пакетов в среде Mininet, исследование влияния данных параметров на надёжность и производительность сетевого соединения.

Проведение интерактивных и воспроизводимых экспериментов с использованием **ping** и **iPerf3**, автоматизация настройки сетевой топологии, измерения RTT, анализа потерь пакетов и повторных передач TCP с помощью Python и Makefile.

2. 2. Процесс выполнения лабораторной работы

2.1 Запускаю сетевую топологию Mininet

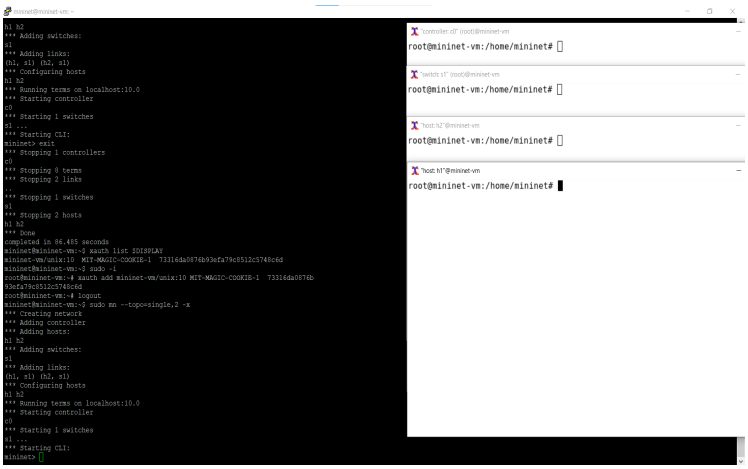


Рисунок 1: Запуск топологии Mininet с двумя хостами и одним коммутатором

2.2 Проверяю сетевые интерфейсы хостов

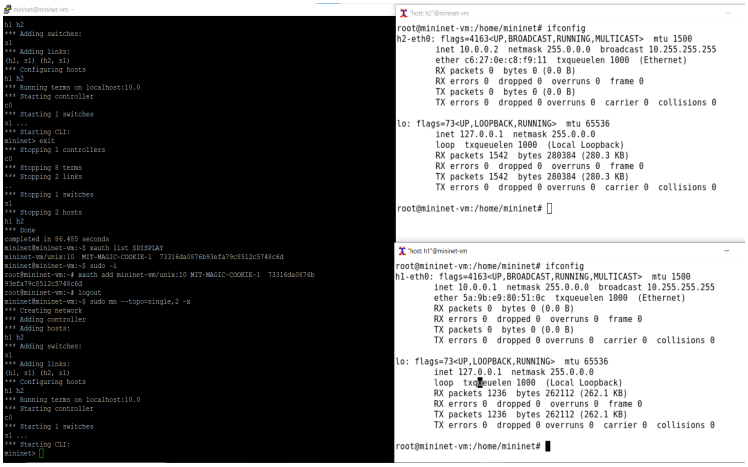


Рисунок 2: Проверка IP-адресов и сетевых интерфейсов хостов h1 и h2

2.3 Измеряю исходные значения RTT

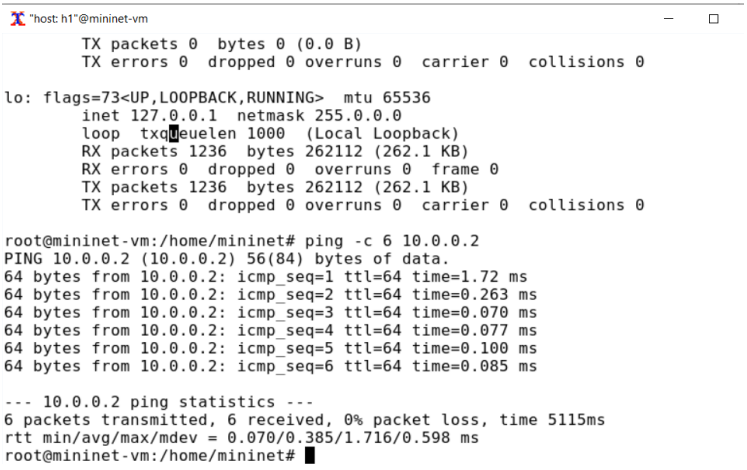


Рисунок 3: Проверка доступности хоста h2 с помощью ping и измерение исходного RTT без потерь пакетов

3. 3. Интерактивные эксперименты NETEM

3.1 Добавляю 10% потерь пакетов на интерфейсе h1

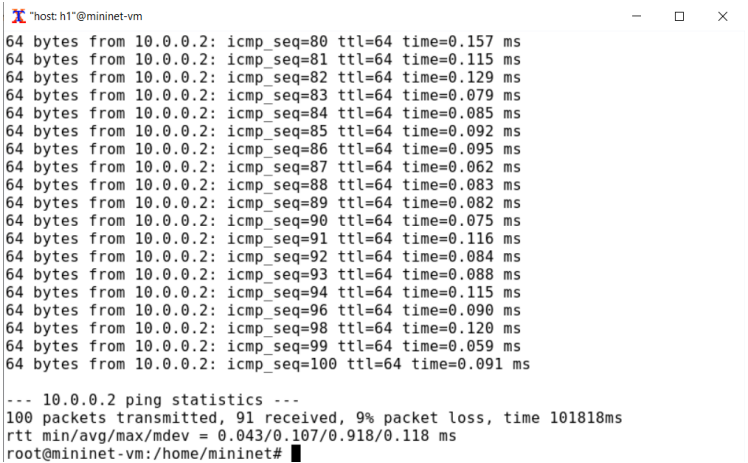


Рисунок 4: Настройка NETEM loss 10% и проверка потерь пакетов с помощью ping

3.2 Настраиваю двунаправленные потери пакетов

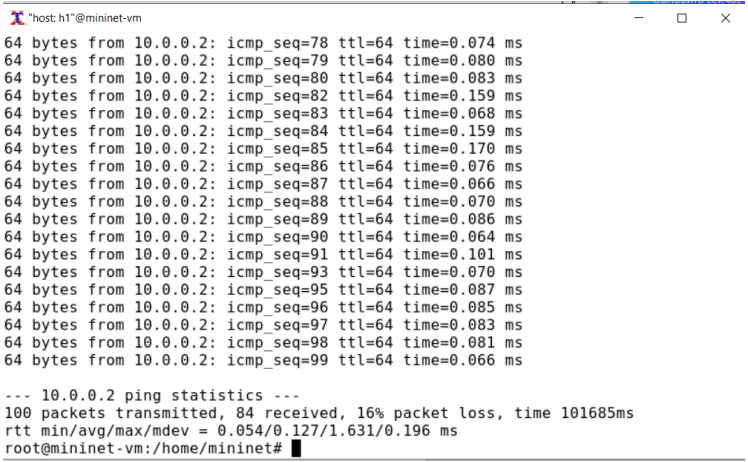


Рисунок 5: Добавление потерь 10% на интерфейсе h2 и измерение суммарных потерь 16%

3.3 Восстанавливаю исходные параметры сети

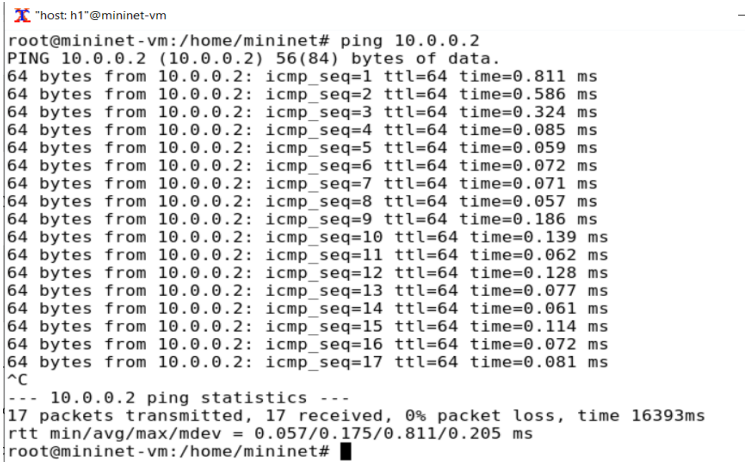


Рисунок 6: Удаление правил NETEM с интерфейсов обоих хостов и проверка отсутствия потерь

3.4 Исследую коррелированные потери пакетов

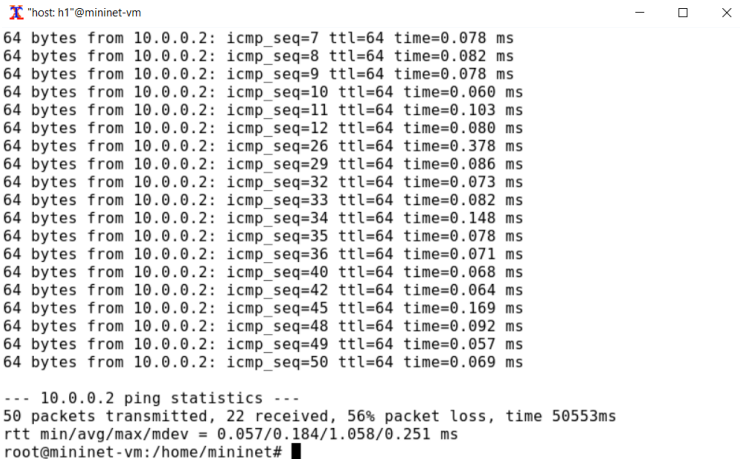


Рисунок 7: Настройка NETEM loss 50% 50% и измерение потерь 56% при передаче ICMP-пакетов

3.5 Исследую повреждение пакетов с помощью iPerf3

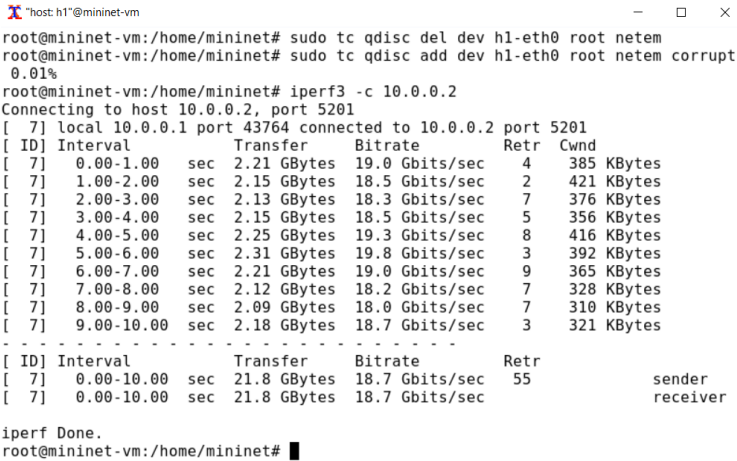


Рисунок 8: Настройка NETEM corrupt 0.01% и измерение повторных передач TCP-сегментов

3.6 Настраиваю переупорядочивание пакетов

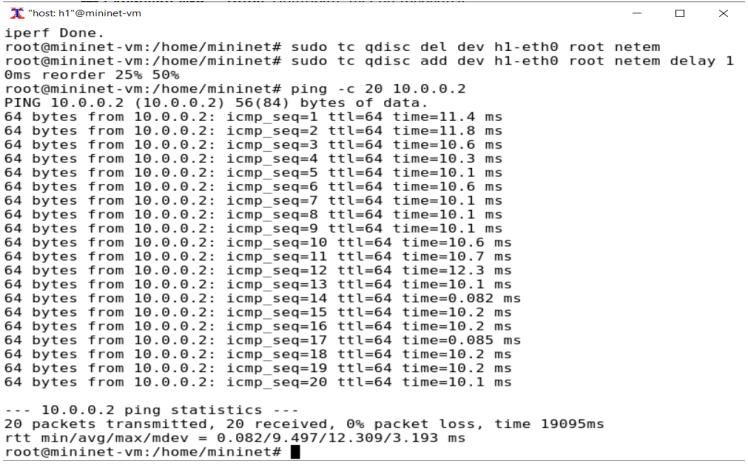


Рисунок 9: Исследование NETEM delay 10ms reorder 25% 50% и измерение времени RTT

3.7 Исследую дублирование сетевых пакетов

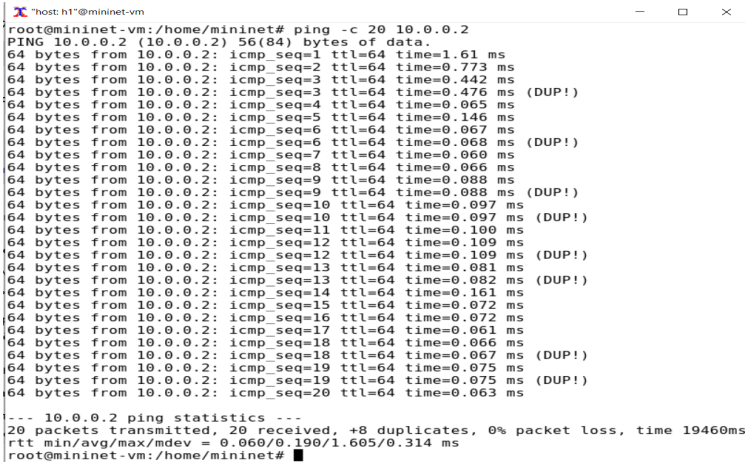


Рисунок 10: Настройка NETEM duplicate 50% и обнаружение повторных ICMP-ответов с отметкой DUP

4. 4. Автоматизация сетевых экспериментов

4.1 Подготавливаю Python-скрипт для эксперимента simple-drop

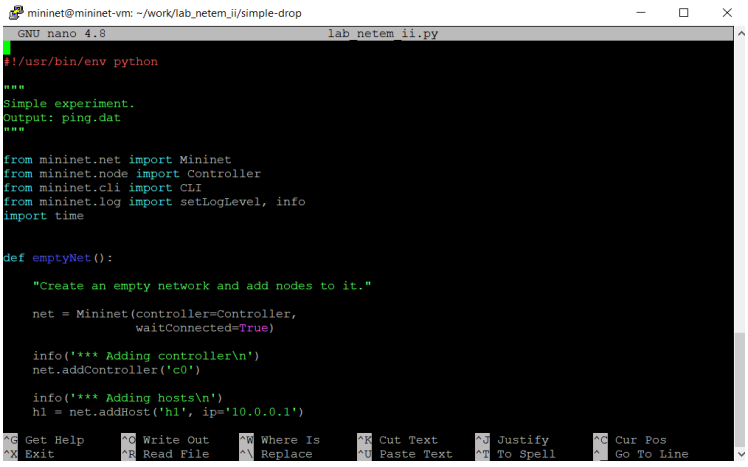


Рисунок 11: Создание скрипта lab_netem_ii.py для автоматизированного исследования потерь пакетов

4.2 Настраиваю топологию и сохранение результатов

mininet@mininet-vm: ~/work/lab_netem_ii/simple-drop

GNU nano 4.8 lab_net

```
    net = Mininet(controller=Controller,
                  waitConnected=True)

    info('*** Adding controller\n')
    net.addController('c0')

    info('*** Adding hosts\n')
    h1 = net.addHost('h1', ip='10.0.0.1')
    h2 = net.addHost('h2', ip='10.0.0.2')

    info('*** Adding switch\n')
    s1 = net.addSwitch('s1')

    info('*** Creating links\n')
    net.addLink(h1, s1)
    net.addLink(h2, s1)

    info('*** Starting network\n')
    net.start()

    info('*** Set delay\n')
    h1.cmdPrint('tc qdisc add dev h1-eth0 root netem loss 10%')
    h2.cmdPrint('tc qdisc add dev h2-eth0 root netem loss 10%')

    time.sleep(10)  # Wait 10 seconds

    info('*** Ping\n')
    h1.cmdPrint(
        'ping -c 100',h2.IP(),
        '| tee ping_full.dat | grep "time=" |'
        ' awk \'{print $5, $7}\' | '
        'sed -e \'s/time=//g\' -e \'s/icmp_seq=//g\' > ping.dat'
    )

    h1.cmdPrint('tail -n 3 ping_full.dat')

    info('*** Stopping network')
    net.stop()
```

Рисунок 12: Реализация настройки потерь NETEM на обоих хостах и записи результатов ping в файлы

4.3 Создаю Makefile для управления экспериментом

mininet@mininet-vm: ~/work/lab_netem_ii/simple-drop

```
  GNU nano 4.8

all: ping.dat

ping.dat:
        sudo python lab_netem_ii.py
        sudo chown mininet:mininet ping.dat

clean:
        -rm -f *.dat
```

Рисунок 13: Подготовка Makefile для автоматического запуска Python-скрипта и очистки результатов

4.4 Запускаю воспроизводимый эксперимент simple-drop

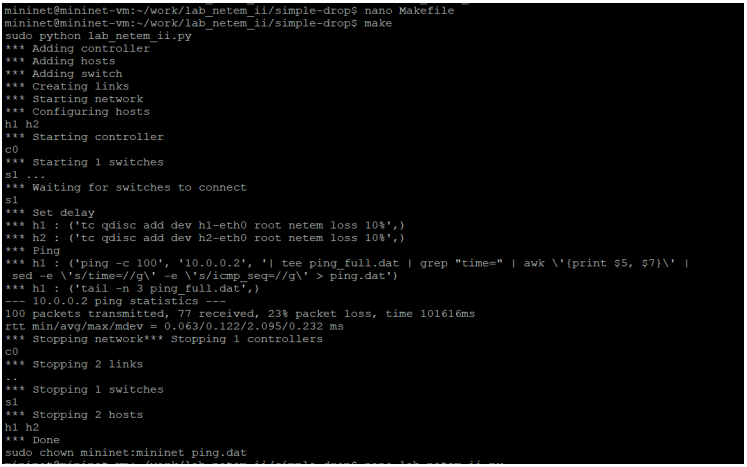


Рисунок 14: Автоматический запуск simple-drop через make и получение статистики потерь пакетов 23%

5. 5. Исследование коррелированных потерь

5.1 Создаю Python-скрипт correlation-drop

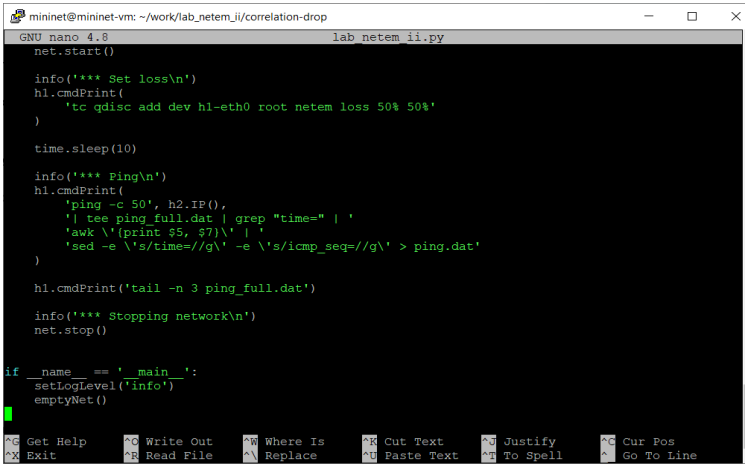


Рисунок 15: Подготовка скрипта для автоматизированного эксперимента с потерями 50% и корреляцией 50%

5.2 Подготавливаю Makefile для correlation-drop

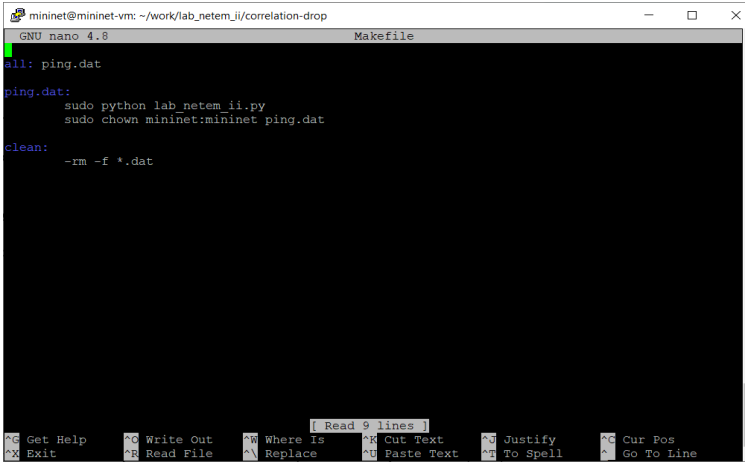


Рисунок 16: Создание Makefile для запуска эксперимента с коррелированными потерями пакетов

5.3 Анализирую результаты correlation-drop

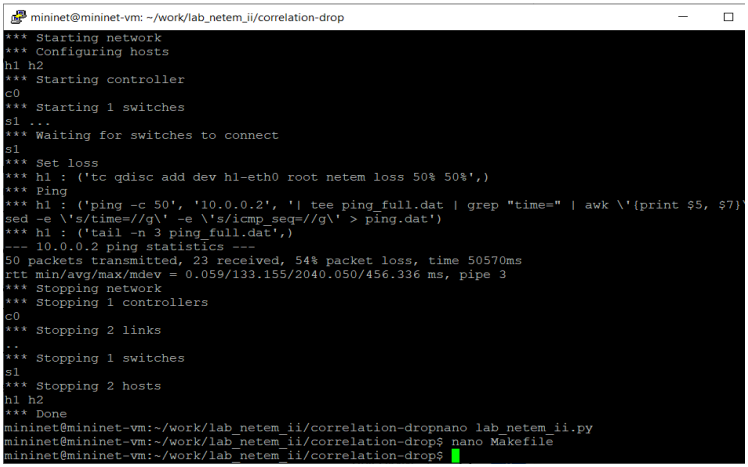


Рисунок 17: Результаты воспроизводимого эксперимента с коррелированными потерями пакетов 54%

6. 6. Исследование переупорядочивания пакетов

6.1 Создаю Python-скрипт для эксперимента reorder

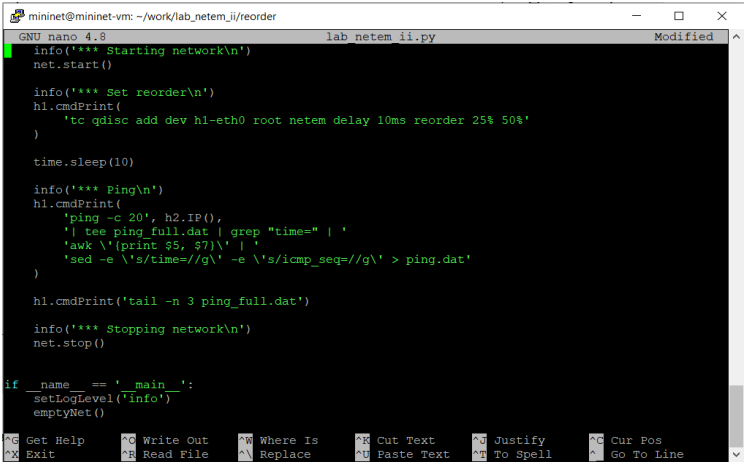


Рисунок 18: Настройка NETEM delay 10ms reorder 25% 50% в автоматизированном Python-скрипте

6.2 Создаю Makefile для эксперимента reorder

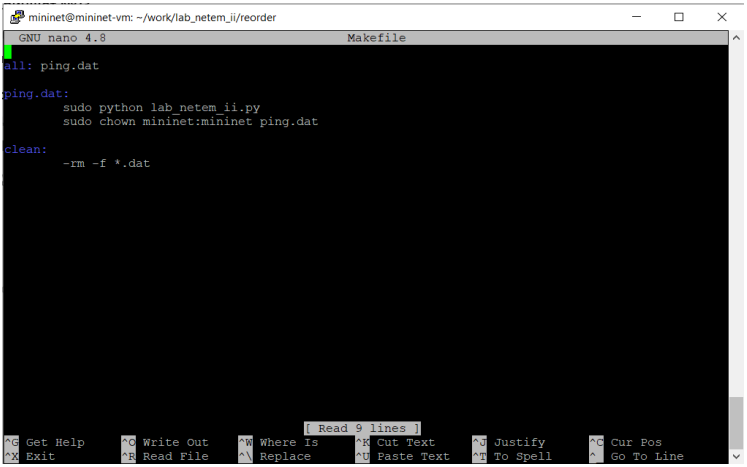


Рисунок 19: Подготовка Makefile для автоматического запуска эксперимента с переупорядочиванием пакетов

6.3 Анализирую результаты эксперимента reorder

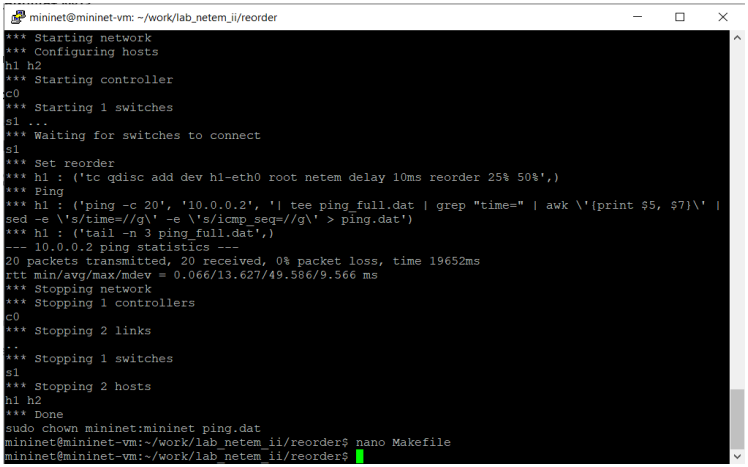


Рисунок 20: Результаты измерения RTT при настройке переупорядочивания пакетов без потерь данных

7. 7. Исследование повреждения пакетов

7.1 Подготавливаю Python-скрипт для эксперимента corrupt

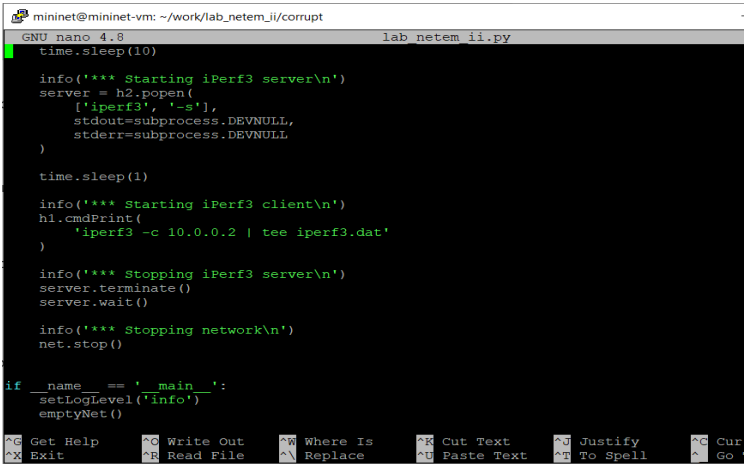


Рисунок 21: Создание скрипта для автоматизированного исследования повреждения пакетов с помощью iPerf3

7.2 Настраиваю Makefile для эксперимента corrupt

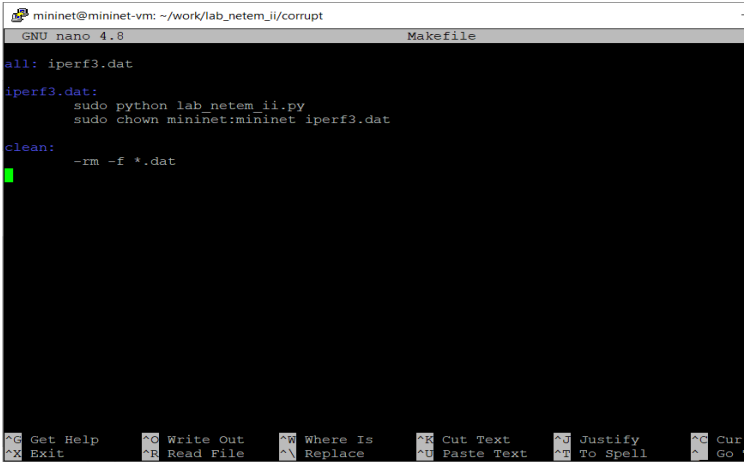


Рисунок 22: Подготовка Makefile для запуска iPerf3 и сохранения результатов в файл iperf3.dat

7.3 Анализирую повторные передачи TCP-сегментов

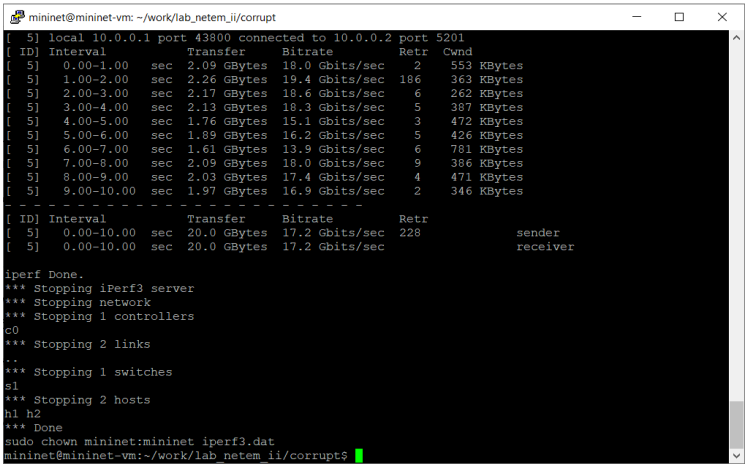


Рисунок 23: Результаты автоматизированного эксперимента с повреждением пакетов и 228 повторными передачами TCP

8. 8. Выводы по проделанной работе

В ходе лабораторной работы изучены возможности **NETEM** для эмуляции потерь, повреждения, дублирования и переупорядочивания пакетов в среде Mininet. Создана сетевая топология из двух хостов **h1** и **h2** и коммутатора **s1**.

Проведены интерактивные эксперименты по настройке вероятности потерь, корреляции, повреждения, задержки и дублирования пакетов. С помощью **ping** измерены значения RTT и процент потерянных пакетов, а инструмент **iPerf3** использован для анализа пропускной способности и повторных передач TCP-сегментов.

С использованием API Mininet, Python и Makefile автоматизированы создание сетевой топологии, настройка NETEM, выполнение экспериментов и сохранение результатов измерений.